CN @ SCALER · WEEK 8 OF 8

# Streaming video at large scale

It starts with a live cricket scorecard.

```
GET /match/412/score.json       → 200 · 1,088 bytes · every 15 s
GET /stream_1/index.m3u8        → 200 ·   228 bytes · every 4 s
GET /stream_1/seg_000008.ts     → 200 · 823,440 bytes · once
```

COURSE MAP

# Eight weeks, one system

WEEK 1
**Network programming 101**

WEEK 2
**Eagle eye view**

WEEK 3
**Server-side considerations for scale**

WEEK 4
**nginx deep dive**

WEEK 5
**Evolution of HTTP**

WEEK 6
**(Ab)using CDNs**

WEEK 7
**Building for failures**

WEEK 8 · TODAY
**Streaming video at large scale**

Today every one of them shows up in a single system: one live match on fifty million phones.

AGENDA

# Today

01 **The brief**
A scorecard for 50M phones, and every way to deliver it

02 **What it costs**
Held sockets, Little's Law, and four bills for one match

03 **Making polling survive**
304s, stacked TTLs, herds, collapsing, negative caching, diffs

04 **Same loop, bigger bytes**
HLS playlists, segments and 188-byte packets

05 **Buffers and latency**
The buffer, the latency budget, spoilers, and ABR

06 **Many origins, many CDNs**
Steering, stale playlists, and cache rules

07 **Making the stream**
Two ffmpeg commands, run for real

08 **Closing**
The course in one match, four ideas, assignment, homework

01 · THE BRIEF

↩ Week 5 · gzip: 202,000 → 5,763 bytes

# One small file, fifty million times

**25–45 s**

between balls; an over every 4–5 minutes

**50 M**

peak concurrent viewers; 20 M on average across a 4-hour match

**~1 KB**

per scorecard: 4–5 KB of JSON, about 1 KB gzipped

**100**

events a year, each one the same file downloaded again and again

↩ Week 5 · HTTP/1.0 hung up after every response

# Two families of answers

NON-PERSISTENT

## The client asks

**Polling:** GET the same URL every few seconds. The connection may close in between.

**Conditional GET**: 304 when nothing changed

**Caches**: nginx in front, then a CDN

**Any server**: no memory of who asked last time

Every request is an ordinary GET: stateless servers, cacheable answers, and a CDN can stand in front.

PERSISTENT

## The server tells

**Long poll**: a GET the server answers only when there is news

**Server-Sent Events**: one response that never ends

**WebSockets**: HTTP upgrades into a two-way, framed pipe

**PubSub (MQTT)**: topics, a broker, fan-out to every subscriber

**Plain binary socket**: your own protocol, like the Week 5 course project

Every viewer holds a connection for the whole match: stateful servers, and nothing a CDN can cache.

↩ Week 7 · full jitter

# Polling: ask again every few seconds

```
let etag = null;
const EVERY = 15_000;  // ms

async function poll() {
  const res = await fetch("/match/412/score.json", {
    headers: etag ? { "If-None-Match": etag } : {},
  });
  if (res.status === 200) {  // something changed
    etag = res.headers.get("ETag");
    render(await res.json());
  }  // 304 Not Modified: keep what we have
  const jitter = 0.8 + Math.random() * 0.4;  // ±20%
  setTimeout(poll, EVERY * jitter);
}
poll();
```

### ETag + If-None-Match

Nothing changed? 304 and an empty body. The request still happens.

### Jitter

±20%, so fifty million phones do not poll on the same second.

### Freshness

At worst one poll interval plus every cache TTL on the path.

↩ Week 3 · Little's Law: hold longer, hold more

# Long polling: ask once, wait for news

```
GET /match/412/score?after=1841 HTTP/1.1

  … the server holds the request open …
  … until ball 1842 is bowled, or 25 s pass

HTTP/1.1 200 OK
{"version":1842,"over":"15.4","total":"146/4"}

GET /match/412/score?after=1842 HTTP/1.1
  … and ask again at once
```

**A held request is a held socket.** Every viewer has one open almost all the time: W is 25 s, not 30 ms.

**Beat every idle timeout.** Answer before nginx's proxy_read_timeout and the ALB idle timeout; both default to 60 s (Week 4).

**It synchronises the herd.** A wicket completes 20M responses in one second, and 20M new requests arrive in the next (Week 7).

**It came back, standardised.** Low-Latency HLS's blocking playlist reload is long polling (block 05).

01 · THE BRIEF

↩ Week 2 · framing: length or delimiter

# Server-Sent Events: one response that never ends

```
GET /match/412/events HTTP/1.1
Accept: text/event-stream
Last-Event-ID: 1841

HTTP/1.1 200 OK
Content-Type: text/event-stream

retry: 5000

id: 1842
event: ball
data: {"over":"15.4","runs":4,"total":"146/4"}

id: 1843
event: ball
data: {"over":"15.5","runs":0,"total":"146/4"}
```

**Framing by delimiter.** An event ends at a blank line: SMTP's lone dot in a different costume.

**Resume from a version.** On reconnect the browser sends Last-Event-ID, and the server replays only what was missed.

**About six per browser.** Over HTTP/1.1 each tab holds its own connection; HTTP/2 multiplexes them onto one (Week 5).

**One way only.** Server to client. Anything the phone says goes in ordinary requests.

# WebSockets: leave HTTP after one exchange

```
GET /live HTTP/1.1
Host: score.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

The Accept value is base64(SHA-1(key + a fixed GUID)): base64 again (Week 1). Client frames are masked so stray bytes can never look like a request to a caching proxy (Week 6).

**After the 101: every frame starts like this**

| Field | Bytes | What it says |
|---|---|---|
| FIN · RSV · opcode | 1 | last fragment? text, binary, ping, pong, close |
| MASK · length | 1 | 0–125 is the length; 126 or 127 means read more |
| extended length | 0, 2 or 8 | a 16- or 64-bit length |
| masking key | 0 or 4 | present on every client-to-server frame |
| payload | length | your 183-byte scorecard diff |

A length, not a delimiter: the other answer from Session 2.

↩ Week 1 · type, length, value

# MQTT: publish once, fan out to everyone

```
phone ↔ broker
→ CONNECT    client=phone-8812 keepalive=60
← CONNACK    accepted
→ SUBSCRIBE  match/412/score qos=1
← SUBACK
← PUBLISH    match/412/score retain=1 {"v":1842,…}
→ PINGREQ    C0 00  two bytes, every keepalive
← PINGRESP   D0 00
```

One PUBLISH from the scorer; the broker copies it to every subscriber of the topic. Hotstar picked MQTT with the EMQX broker for exactly this fan-out.

**Retained message.** The broker keeps the last score and hands it to each new subscriber at once: a cache, inside the broker.

**QoS 1 is at-least-once.** Duplicates happen; apply updates by version number and they become harmless (Week 7).

**Remaining Length is a varint.** 7 bits per byte, the top bit says "more": a type, a length, a value.

**Keepalives cost.** Two bytes, but they wake the phone's radio and keep a NAT mapping alive.

01 · THE BRIEF

↩ Week 3 · from C10K to C10M

# How Hotstar held 50 million sockets

1

### One node

EMQX on one c5.4xlarge: 250k connections, latency in bounds

2

### A cluster

Publisher split from subscribers: 2M subscribers. Mnesia wants fewer than 10 nodes.

3

### The ELB wall

Drops at ~500k without pre-warming; 4M per pre-warmed Classic ELB

4

### Five clusters

5 clusters × 8 subscribe nodes, bridged: past 10M

5

### Reverse bridge

A Go service on each subscriber pulls from the publisher. 200 nodes: 50M

> *“Caching at any level is also not possible.”*
>
> Their social feed was tied to live moments, so it had to be pushed. A scorecard is the opposite case: one file, identical for everyone.

Source: M. Chaudhari and P. Gupta, "Building Pubsub for 50M concurrent socket connections", Hotstar Engineering blog, 6 June 2019

↩ Week 3 · nothing is free per connection

# What a held socket costs

### Machines

Stateful: each phone lives on one box for hours. 100k–200k connections per box on our board; Hotstar measured 250k. New boxes sit empty until someone reconnects, so rebalancing means disconnecting people.

### Memory

Kernel socket buffers, TLS state, the broker's per-client process. 250k phones on a 32 GiB box is about 137 KB each, all in. And the soft ulimit -n still starts at 1,024.

### Network

Keepalives with nothing to say: at a 60 s keepalive, 20M idle phones send 333k pings a second before the first ball is bowled.

### Load balancers

One address instead of 200 for DNS to hand out, and an even spread across boxes. But the LB holds every socket too: Hotstar hit 4M per Classic ELB.

Testing it: one client IP has 28,232 ephemeral ports (32768–60999) per destination, so 250k test sockets need at least 9 source IPs. Week 1: a port is 16 bits.

02 · WHAT IT COSTS

↩ Week 3 · L = λ × W

# Little's Law, twice

POLLING THROUGH A CDN

1.67 M requests/s × about 0.03 s at the edge

**50,000**

requests in flight

PUSH OVER HELD SOCKETS

W is the whole match: four hours

**50,000,000**

sockets held, ball or no ball

Same audience, same scorecard: a thousand times the concurrency, because W went from milliseconds to hours.

↩ Week 4 · nginx serves it from memory

# Where the scorecard file lives



↩ Week 3 · multiply before you choose

# The arithmetic before the architecture

**9.6 B**
requests per match

**1.67 M**
requests a second at the peak

**9.6 TB**
of scorecard per match

```
requests / match = 20M viewers × 14,400 s ÷ 30 s = 9.6 B
peak rate        = 50M viewers ÷ 30 s            = 1.67 M req/s
bytes / match    = 9.6 B × 1 KB                  = 9.6 TB
poll every 15 s → 19.2 B requests, 3.33 M req/s, 19.2 TB
push each ball  → about 9.6 B messages, one every ~30 s per phone
× 100 events    → 960 B requests and 960 TB a year
```

↩ Week 6 · a CDN is a rented reverse proxy

# Four ways to pay for one match

| Design | Per match | × 100 events | Mostly paying for | Breaks first |
|---|---|---|---|---|
| S3 straight to phones | $4.7K | $470K | GET fees ($3,840) | 5,500 GET/s per prefix |
| nginx fleet + ALB | $1.2–1.8K | $120–180K | egress ($864) + load balancer units | 20–50M idle keep-alive sockets at the LB |
| CDN (CloudFront, India) | $12.5K | $1.25M | request fees: 92% of it | nothing; it is just the bill |
| Push: 200 × c5.4xlarge | $0.9–1.6K | $90–160K | a peak-sized fleet ($680) + egress | reconnect storms, rebalancing |

At list price, push is the cheapest per match and the CDN the dearest. Hold that thought.

AWS list prices, Sep 2026 · S3, EC2, ELB: US East · CloudFront: India · body bytes only · poll every 30 s · push sends 183-byte diffs up to full cards

↩ Week 7 · 503 and Retry-After

# S3 runs out of requests before money

**5,500**
GETs a second that one S3 prefix promises, at least

**1.67 M**
GETs a second that 50M phones polling every 30 s ask for

**303×**
over, and one key lives under one prefix

> *“While Amazon S3 is scaling to your new higher request rate, you may see some 503 (Slow Down) errors.”*
>
> Fifty million clients retrying a 503 without backoff is Session 7's retry storm, pointed at your own bucket.

Source: Amazon S3 User Guide, "Best practices design patterns: optimizing Amazon S3 performance" (checked Sep 2026)

02 · WHAT IT COSTS

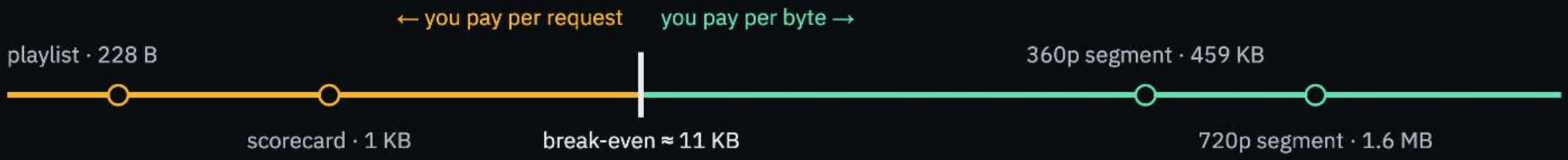
↩ Week 5 · headers: 97.5% repeats

# On a CDN, a 1 KB file is billed per request

**92%**

of the scorecard's CDN bill is request fees, not bytes

| Per million responses | Request fee | Byte fee |
|---|---|---|
| Scorecard, 1 KB | $1.20 | $0.11 |
| 480p segment, 823 KB | $1.20 | $59–90 |




CloudFront India list prices, Sep 2026: HTTPS $0.0120 per 10,000 requests · $0.109/GB after 1 free TB, $0.072/GB past 5 PB · log scale

↩ Week 3 · HTTP is stateless, the rest aren't

# “I prefer polling because I want to tell you how it’s used for video too.”

### Cacheable

One file, identical for fifty million people.

### Stateless

Any server, any edge, any CDN can answer any phone.

### Already paid for

At list price the scorecard is 0.28% of the same match's video bill.

↩ Week 5 · ETag: a fingerprint, not a clock

# Conditional GET: 304 Not Modified

```
GET /match/412/score.json HTTP/1.1
If-None-Match: "v1841"

HTTP/1.1 304 Not Modified
ETag: "v1841"
Cache-Control: public, max-age=10
```

Make the ETag the scorecard's version number: a fingerprint, not a clock. If-Modified-Since works too, but only to the second.

SAVES

The 1 KB body, the JSON parse, the re-render

DOES NOT SAVE

The request fee, the round trip, the headers

PAYS MOST BETWEEN CDN TIERS

Edge to mid-tier, mid-tier to origin: 304s instead of copies

↩ Week 6 · tiered cache: edge, then mid-tier

# TTLs add up

Every layer set to 30 s




Up to 90 s stale: two or three balls behind the stadium

Each CDN TTL at half the promise, or less




Up to 25 s: inside one ball

Worst-case staleness = the poll interval + every cache TTL on the path. Keep each CDN TTL at ½ of the freshness you promise or less, and refresh the edge before it expires: a prefetch at ~25 s of a 30 s TTL (Akamai calls it prefresh).

03 · MAKING POLLING SURVIVE

↩ Week 4 · proxy_cache_lock + use_stale

# The thundering herd at expiry

At 10.000 s the edge's copy expires. Every request in that instant is a miss, and all of them head for the origin.

### Request collapsing

One fetch per edge per object; everyone else waits in a queue.

### Serve stale while updating

Answer from the old copy while one request refreshes it (RFC 5861).

### Prefetch before expiry

Refresh at ~25 s of a 30 s TTL, so no viewer meets an empty cache.

### Jitter from the client

Spread polls across the interval, not on the same second (Week 7).

```
proxy_cache_lock              on;
proxy_cache_use_stale         updating error timeout http_500 http_502 http_503 http_504;
proxy_cache_background_update on;
proxy_cache_valid 200         10s;
```

03 · MAKING POLLING SURVIVE

↩ Week 5 · 103 Early Hints replaced push

# Request collapsing needs a shareable answer

**20,000 phones**
ask one edge for score.json in the same 50 ms

→

**edge**
sends 1 request upstream; 19,999 wait in a queue

→

**origin**
sees 1 request

## Make it public early

The first response decides. Cache-Control: public releases every waiter at once. A private answer, or a stray Set-Cookie, dequeues them one by one, and Fastly then skips collapsing for that object for 2 minutes.

## Early hints

Name the next thing before it exists: HTTP 103 for pages; in Low-Latency HLS, a preload hint names the next piece, requests for it queue at the edge, and one arrival releases them all.

Check the default first: Cloudflare does not cache .json out of the box (Week 6). Without a cache rule, cf-cache-status says DYNAMIC and every poll reaches you.

# Negative caching: 404 and 5xx

### 5XX · CACHE IT, BRIEFLY

One or two seconds of a cached 503 shields a struggling origin from fifty million retries.

The edge answers; the origin sees one probe per TTL per edge.

### 404 · ALMOST NEVER

A 404 for a file that is about to exist keeps being served until its TTL runs out.

The next match's scorecard, the newest live segment. Cloudflare's default edge TTL for a 404 is 3 minutes.

```
proxy_cache_valid 200             10s;
proxy_cache_valid 404              1s;
proxy_cache_valid 500 502 503 504  2s;
```

Rule: a negative TTL must be far shorter than the time it takes the missing thing to appear.

03 · MAKING POLLING SURVIVE

↩ Week 6 · Railgun → RFC 9842 deltas

# Send diffs, not the whole card

**4,117 B**
the full scorecard JSON

**1,088 B**
the same card, gzipped

**183 B**
one ball as a gzipped JSON Patch

Measured on a synthetic T20 card: two innings, batting and bowling figures, fall of wickets.

### Push: send the patch

SSE or MQTT carry the 183-byte patch. A phone that missed one sees a version gap and fetches the full card once.

### Pull: version the file

?since=1841 gives every client its own URL and the hit rate collapses. Publish one file with the version and the last few changes instead.

↩ Week 2 · sequence numbers make streams resumable

# A versioned scorecard is a playlist

score.json · polled every 15 s

```
{
  "version": 1842,
  "recent": [
    {"v": 1839, "ball": "15.1", "runs": 1},
    {"v": 1840, "ball": "15.2", "runs": 0},
    …
    {"v": 1842, "ball": "15.4", "runs": 4}
  ]
}
```

index.m3u8 · polled every 4 s

```
#EXTM3U
#EXT-X-TARGETDURATION:4
#EXT-X-MEDIA-SEQUENCE:2
#EXTINF:4.000000,
seg_000002.ts
…
#EXTINF:4.000000,
seg_000011.ts
```

version ↔ MEDIA-SEQUENCE · recent changes ↔ the last N segments · poll interval ↔ TARGETDURATION

A small index everyone polls, pointing at numbered pieces that never change. That design has a name: HTTP Live Streaming.

↩ Part 1 · the scorecard loop

# Same loop, bigger bytes

**1 KB**
scorecard, polled every 15 s

**228 B**
video playlist, polled every 4 s

**823 KB**
one 4 s segment at 480p, fetched once

DEMO

Open Big Buck Bunny or Tears of Steel on videojs.org with DevTools → Network open.

Watch the master playlist, then a media playlist, then segment after segment. Throttle the network and watch it change rendition.

↩ Week 2 · control and data want separating

# HLS in one picture



↩ Week 7 · a codec fallback is not a retry

# The master playlist

```
#EXTM3U
#EXT-X-VERSION:6
#EXT-X-STREAM-INF:BANDWIDTH=3440800,RESOLUTION=1280x720,
  CODECS="avc1.4d401f,mp4a.40.2"
stream_0/index.m3u8

#EXT-X-STREAM-INF:BANDWIDTH=1755600,RESOLUTION=854x480,
  CODECS="avc1.4d401f,mp4a.40.2"
stream_1/index.m3u8

#EXT-X-STREAM-INF:BANDWIDTH=950400,RESOLUTION=640x360,
  CODECS="avc1.4d401e,mp4a.40.2"
stream_2/index.m3u8
```

From our ffmpeg run, attribute lines wrapped. BANDWIDTH = (video + audio) × 1.1: (3000k + 128k) × 1.1 = 3,440,800.

**"Do not poll this again"**

It does not change during the match: fetch once, cache for minutes.

**"Some people poll: switch CDNs"**

Re-reading the menu is how a player learns a new path. Standardised later as Content Steering (block 06).

**"Falling back to another codec"**

CODECS lets the player skip what it cannot decode, say HEVC down to H.264. A fallback, not a retry.

↩ Part 1 · the scorecard, every 4 s

# The media playlist, polled

```
# reload at t = 47.0 s
#EXT-X-TARGETDURATION:4
#EXT-X-MEDIA-SEQUENCE:1
#EXTINF:4.000000,
#EXT-X-PROGRAM-DATE-TIME:…T03:16:12.365+0000
seg_000001.ts
  … 8 more …
#EXTINF:4.000000,
#EXT-X-PROGRAM-DATE-TIME:…T03:16:48.365+0000
seg_000010.ts
```

```
# reload at t = 51.5 s
#EXT-X-TARGETDURATION:4
#EXT-X-MEDIA-SEQUENCE:2
#EXTINF:4.000000,
#EXT-X-PROGRAM-DATE-TIME:…T03:16:16.365+0000
seg_000002.ts
  … 8 more …
#EXTINF:4.000000,
#EXT-X-PROGRAM-DATE-TIME:…T03:16:52.365+0000
seg_000011.ts
```

**Changed?** Wait at least one target duration before reloading again.

**Unchanged?** Wait half a target duration, then try again.

**Same number, different URI?** “It SHOULD halt playback.”

RFC 8216 §6.3.4 · two reloads of our live 360p playlist, 4.5 s apart, trimmed and with the dates elided

04 · SAME LOOP, BIGGER BYTES

↩ Week 6 · the query string is in the cache key

# VOD is a playlist that stopped changing

```
#EXTM3U
#EXT-X-TARGETDURATION:10
#EXT-X-MEDIA-SEQUENCE:0
#EXTINF:10,
1719543778001_4328560526001_s-1.ts?pubId=1719543778001&videoId=4328294689001
#EXTINF:10,
1719543778001_4328560526001_s-2.ts?pubId=1719543778001&videoId=4328294689001
  … 61 more, 10 s each …
#EXTINF:5,
1719543778001_4328560526001_s-64.ts?pubId=1719543778001&videoId=4328294689001
#EXT-X-ENDLIST
```

**#EXT-X-ENDLIST**

No more segments: read it once, stop polling.

**63 × 10 s + 5 s**

10 s segments: old advice. Apple now says 6 s.

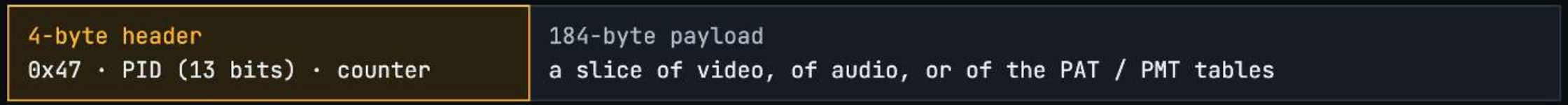
**?pubId=…&videoId=…**

Same for everyone, so it caches. A per-viewer token would not.

04 · SAME LOOP, BIGGER BYTES

↩ Week 2 · why 140 bytes? why 188?

# A segment is 188-byte packets




| Rendition | One segment | 188-byte packets | Overhead |
|---|---|---|---|
| 720p | 1,613,416 B | 8,582 | 3.3% |
| 480p | 823,440 B | 4,380 | 4.1% |
| 360p | 452,328 B | 2,406 | 5.6% |

Measured on our run: every segment is an exact multiple of 188. Overhead is everything that is not audio or video payload.

**Fixed length and a sync byte.** Both of Session 2's framing answers at once. 188 was chosen for ATM.

**A 13-bit PID says whose packet it is.** Audio, video and tables share one byte stream (Week 5).

**fMP4 and CMAF** segments are boxes instead: size, type, payload. TLV again (Week 1).

# How long should a segment be?

| Segment | Peak segment requests | Latency floor (3 ×) | On our board |
|---|---|---|---|
| 2 s | 25 M/s | 6 s | |
| 3 s | 16.7 M/s | 9 s | "RPS?" |
| 4 s | 12.5 M/s | 12 s | our ffmpeg run |
| 6 s | 8.3 M/s | 18 s | Apple's advice |
| 30 s | 1.7 M/s | 90 s | "Too late!" |

**ADAPTIVE / HYBRID**

1–2 s segments at start-up for a fast first frame, longer ones later. ffmpeg has a knob for it: -hls_init_time 1 -hls_time 4.

**THE TRADE**

Shorter: more requests, more keyframes, more round trips. Longer: more latency, slower reaction to bandwidth.

↩ Part 1 · the scorecard problem, again

# Playlist traffic is harder than segment traffic

| | Media playlist | Segment |
|---|---|---|
| Size | ~230 B gzipped | 450–1,600 KB |
| Changes | every 4 s | never |
| Cache TTL | ½ target duration or less | minutes to forever |
| A stale copy | stalls, or points at deleted files | harmless |
| Billed by | requests, like the scorecard | bytes |

Board: "Rendition: poll every second → RPS?" At 1 s that is 50 M requests a second at the peak. At the RFC's one target duration, 12.5 M.

↩ Week 7 · the buffer is the stale cache

# The buffer




The amber line is the playhead: 12 s buffered ahead of it, the next segment still downloading.

### Large buffer

Survives a bad minute on a train, but costs latency, and memory on low-end phones and TV sticks.

### One-segment buffer

Lowest latency. One slow segment and the picture stalls.

### Network bandwidth

More network, less buffer: on a fast link a refill takes a fraction of a segment, so a small buffer is enough.

### Stale playlist

When the playlist stops moving, the buffer is what keeps the picture going.

↩ Week 5 · server push died (Chrome 106)

# Latency is a budget

| encode | segment fills | to CDN | player hold-back: 3 × target duration |
|---|---|---|---|
| ~2 s | 4 s | ~2 s | 12 s |

About 20 s behind the stadium with 4-second segments. Encode and CDN times are typical guesses; the hold-back is the spec.

Low-Latency HLS shrinks the biggest bar with four ideas from the first half:

**Parts**
~1 s pieces of a segment. Apple: at least 3× the P95 round trip (Week 6).

**Blocking reload**
?_HLS_msn=… is held until that segment exists. Long polling.

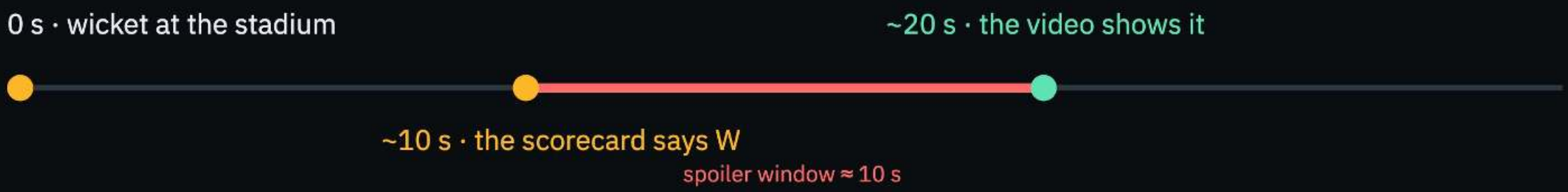
**Preload hints**
Name the next part early. Replaced HTTP/2 push in Jan 2020.

**Delta updates**
EXT-X-SKIP drops old entries from the reply: diffs, for playlists.

With 1-second parts the 12 s hold-back becomes about 3 s: Apple requires PART-HOLD-BACK of at least three parts.

↩ Part 1 · two polling loops, one clock

# The spoiler problem




### Hold the score until the picture catches up

Stamp each update with the ball's wall-clock time. The player knows the wall-clock time of every segment, so the app reveals an update only when playback passes it.

```
# the score update
{"v": 1842, "ball": "15.4",
 "at": "2026-09-27T03:16:44.1Z"}
# the segment on screen
#EXT-X-PROGRAM-DATE-TIME:
2026-09-27T03:16:44.365+0000
```

↩ Week 7 · who decides “worse”?

# ABR: the player picks the next rendition

> *Last week: every fallback gives you something worse than ideal. When does worse stop being acceptable, and who decides?*
>
> The player decides, at every segment, under a policy the product sets.

### Throughput-based

Measure how fast the last segments arrived; pick the highest rendition that fits. Congestion control, one layer up.

### Buffer-based

Watch the buffer: nearly empty, step down; full, step up. Netflix's A/B tests: 10–20% fewer rebuffers at a similar video rate.

**The policy:** a stall is worse than a lower rendition. Even the default is policy: Apple suggests starting cellular viewers on the 730 kbps variant and Wi-Fi viewers on 2 Mbps.

Huang, Johari, McKeown, Trunnell, Watson, "A Buffer-Based Approach to Rate Adaptation", SIGCOMM 2014 · Apple HLS authoring spec 1.32

↩ Week 6 · Fastly's 49 minutes

# Many origins, many CDNs




### DNS

Hand out a CDN per region or ISP. Slow to change: resolvers cache.

### Listed twice

The master lists each rendition once per CDN, for stream failover.

### Content Steering

A JSON file, polled every TTL, names the preferred CDN.

### Client failover

Errors on one CDN trip a circuit breaker; the next segment tries the other (Week 7).

↩ Week 6 · stale-if-error, with a catch

# Stale playlists and deleted segments

1. ffmpeg's delete_segments keeps one unreferenced segment; we watched seg_000000 vanish 4 s after it left the playlist.
2. RFC 8216 says a removed segment MUST stay for its duration plus the playlist's: 4 + 40 = 44 s here.
3. During an origin blip the CDN serves a stale playlist. It still lists a segment the origin has deleted.
4. 404, negatively cached, and the player stalls. “Serve stale may cause player issues.”

FIXES

`-hls_delete_threshold 12`

Playlist TTL at ½ target duration or less; serve it stale for one target duration at most.

Segments never change: serve them stale freely.

Apple: treat a playlist as invalid when Last-Modified is more than 3 target durations behind Date.

```
# our run, 360p rendition
t = 47.0 s   playlist: seg_000001 … seg_000010   on disk: seg_000000 … seg_000010
t = 51.5 s   playlist: seg_000002 … seg_000011   on disk: seg_000001 … seg_000011
```

↩ Week 4 · location blocks: prefix vs regex

# Caching rules for a live stream

| Object | TTL | Serve stale | 404 |
|---|---|---|---|
| master.m3u8 | minutes | yes | 1 s |
| media playlist | ½ target duration or less | one target duration at most | 1 s, or not at all |
| segment .ts | long: it never changes | yes | 1 s at most, at the live edge |
| score.json | ½ your promise or less | while updating | 1 s (5xx: 2 s) |

```
location ~ \.m3u8$ { proxy_cache_valid 200 2s;  proxy_cache_lock on; proxy_cache_use_stale updating; }
location ~ \.ts$   { proxy_cache_valid 200 10m; proxy_cache_valid 404 1s; }
gzip_types application/vnd.apple.mpegurl application/json;
```

Also: gzip every playlist (Apple: MUST), and keep per-viewer tokens out of the cache key.

↩ Week 4 · the honest benchmark

# From an mp4: the one-liner

```
ffmpeg -re -i input.mp4 -codec: copy -hls_time 4 -hls_list_size 5 \
       -hls_playlist_type event -f hls index.m3u8
```

| Flag | What it does |
|---|---|
| -re | read at native speed: a file pretending to be a camera |
| -codec: copy | no re-encode, so it can only cut at existing keyframes |
| -hls_time 4 | a target segment length, not a promise |
| -hls_list_size 5 | a 5-segment window, unless… |
| -hls_playlist_type event | …the playlist is append-only |

## WHAT RUNNING IT SHOWED

**8.33 s segments, not 4.** x264's default keyframe interval is 250 frames: 8.33 s at 30 fps.

**The window was ignored.** An EVENT playlist only grows: 11 segments listed at the end.

**For real:** camera → RTMP or SRT → transcoder → packager → origin → CDN.

↩ Week 3 · amortise the setup

# The ABR ladder, annotated

```
ffmpeg -re -stream_loop -1 -i input.mp4 \
  -filter_complex "[0:v]split=3[v720][v480][v360]; …scale + pad to 1280x720, 854x480, 640x360…; \
                   [0:a]asplit=3[a0][a1][a2]" \
  -map "[v720o]" -map "[a0]" -map "[v480o]" -map "[a1]" -map "[v360o]" -map "[a2]" \
  -c:v libx264 -preset veryfast -profile:v main -sc_threshold 0 -g 120 -keyint_min 120 \
  -c:a aac -ac 2 -ar 48000 \
  -b:v:0 3000k -maxrate:v:0 3210k -bufsize:v:0 4500k -b:a:0 128k \
  -b:v:1 1500k -maxrate:v:1 1605k -bufsize:v:1 2250k -b:a:1  96k \
  -b:v:2  800k -maxrate:v:2  856k -bufsize:v:2 1200k -b:a:2  64k \
  -f hls -hls_time 4 -hls_list_size 10 \
  -hls_flags independent_segments+delete_segments+omit_endlist+program_date_time \
  -hls_segment_filename "out/stream_%v/seg_%06d.ts" -master_pl_name master.m3u8 \
  -var_stream_map "v:0,a:0 v:1,a:1 v:2,a:2" "out/stream_%v/index.m3u8"
```

**split=3:** decode once, encode three times.

**-g 120:** 4 s at 30 fps only. A 24 fps source gave 5 s segments.

**-hls_flags:** a 40 s window that never ends, stamped with wall-clock time.

08 · CLOSING

# The whole course in one cricket match

**WEEK 1 · SOCKETS**
Ports are 16 bits: 28k test sockets per client IP. Base64 in the WebSocket handshake.

**WEEK 2 · WIRE**
Framing: SSE's blank line, WebSocket lengths, 188-byte packets. Playlists as control, segments as data.

**WEEK 3 · SCALE**
Fifty million held sockets, Little's Law, and nothing free per connection.

**WEEK 4 · NGINX**
The origin: proxy_cache_lock, use_stale, one location per file type.

**WEEK 5 · HTTP**
ETag and 304, keep-alive, 101 Upgrade, and push's death shaping LL-HLS.

**WEEK 6 · CDNS**
Tiers, collapsing, negative caching, and paying per request or per byte.

**WEEK 7 · FAILURES**
Jitter, the live-edge 404, stale-if-error, failover, and who decides “worse”.

**WEEK 8 · VIDEO**
HLS: a playlist, polled like a scorecard, pointing at bytes that never change.

Every week showed up in one match. That was the point of the course.

08 · CLOSING

# Four ideas to keep

1 **Pull scales through caches; push scales through machines.**

The same answer for everyone belongs in a CDN. A different answer per person means you are buying sockets.

2 **Number the pieces, then cache them forever.**

A small changing index plus numbered pieces that never change: the versioned scorecard and the HLS playlist are one design.

3 **Latency is a budget you spend on safety.**

Poll intervals, TTLs, segment length and hold-back all add up. Decide where each second goes.

4 **Know which bill you are on.**

Below about 11 KB you pay per request, above it per byte. Scorecards and playlists are the first kind; segments the second.

↩ Weeks 5–6 · build it, then measure it honestly

# Assignment: a match on your laptop

origin: nginx, score.json, and the ffmpeg ladder ← mid-tier: an nginx cache in front ← 1,000 simulated viewers: pollers and players

### 1 · Collapsing

Origin requests per second for score.json with proxy_cache_lock off, then on.

### 2 · Staleness

For each ball, seen minus written, at a 30 s TTL and at 5 s. Compare with the TTL-stack slide.

### 3 · Break it

Serve a stale playlist with hls_delete_threshold 1 and catch the 404s. Fix it; show the clean log.

### 4 · Spoiler

Show the score ahead of the picture, then hold updates until PROGRAM-DATE-TIME passes them.

Deliverable: code and a one-page write-up with the numbers. Due: [date].

# Homework

1. Play the video.js demo with DevTools open: log two minutes of requests, then throttle and catch a rendition switch.
2. Re-run the four bills for a 15 s poll, and for a negotiated $0.02/GB with no request fee. Which design wins now?
3. Serve the scorecard over SSE on HTTP/1.1, open seven tabs, and explain the one that waits. Then fix it with HTTP/2.
4. Read Hotstar's pubsub post and map each of its five steps to a Week 3 or Week 4 idea.

5 THE ONE TO DO IF YOU DO ONLY ONE

Run the ladder on a 24 fps file and prove from the playlist that the segments are 5 s. Switch to time-based keyframes (force_key_frames) and prove they are 4 s.

CN @ SCALER · WEEK 8 OF 8

# Every live stream is a scorecard with bigger bytes.

Thank you for eight weeks.